

13. Security, AWS

Next.js, GraphQL

Attention!

This training is **interview-driven**.

To become a qualified developer, *ChatGPT* and *YouTube* will always be your best friends.

Outline

- Security
- AWS, S3, CloudFront, Amplify
- [Next.js](#)
- GraphQL

Same-Origin Policy & CORS

- **Same-Origin Policy (SOP)**: a browser security rule that prevents a web page from making requests to a different origin
 - Origin: Protocol(http vs https)+ Domain (google.com) + Port(:80 vs :8080)
 - Under SOP, a web page can only access data from the **same** origin
 - Protect users from Cross-Site Request Forgery (CSRF) and Cross-Site Scripting (XSS)
- **CORS (Cross-Origin Resource Sharing)**: A mechanism that allows servers to opt in to accepting cross-origin requests
 - Server can “white-list” IP address
 - Access-Control-Allow-Origin: app.use(cors())

React Proxy

- In package.json: "proxy": "<http://localhost:4000>"
 - Specifically for development, it avoids needing to configure CORS
- How it works
 - React frontend runs on <http://localhost:3000>
 - Express backend runs on <http://localhost:4000>
 - different ports => different origins (<protocol>://<host>:<port>)
 - Same-Origin Policy => CORS

Scenario	Cross-Origin?	Will it work?
<code>axios.get('/api/files')</code> with proxy	✗ No	✓ Yes
<code>axios.get('http://localhost:4000/api/files')</code>	✓ Yes	✗ Fails without CORS

Https

- Https (HyperText Transfer Protocol Secure): A secure version of HTTP, with a layer of encryption that involves **TLS**(Transport Layer Security)/**SSL**(Secure Sockets Layer) encryption
 - HTTPS = HTTP + SSL/TLS

Feature	HTTP (insecure)	HTTPS (secure)
Data encrypted?	✗ No	✓ Yes
Identity verified?	✗ No	✓ Yes (via certificate)
Data integrity?	✗ No	✓ Yes

Encryption

- Encryption: A process that transforms plaintext into unreadable data using an algorithm and a key
 - Plaintext: human-readable text
- Hashing: A **one-way** function that converts data into a fixed-length string (the "hash")
 - Not reversible
 - Hashing Algorithms: MD5, SHA-1, SHA-2, Bcrypt
- Salt: a random value added to data before it is hashed
 - password hashing
 - $\text{hash}(\text{password} + \text{salt}) \rightarrow \text{stored_value}$
- **bcryptjs**: A JS library that allows secure password hashing using the bcrypt algorithm
 - sign up: `bcrypt.hash(data, saltOrLength, callback)`
 - log in: `bcrypt.compare(plaintext, hashed, callback)`

Authentication & Authorization

- **Authentication:** Who are you
 - Verifying the identity of users accessing the application
- **Authorization:** What are you allowed to do?
 - Controlling access to different resources based on user roles and permissions
 - Role-based access control (**RBAC**)
 - Access Control Lists (**ACLs**)

Feature	Authentication	Authorization
Purpose	Verify identity	Grant or deny access
Happens when?	First (login step)	After authentication
Based on	Credentials (e.g., password)	Roles/permissions (e.g., isAdmin)
Example	Login with email/password	Access to <code>/admin</code> only for admins

Session & Cookie

- **Session:** Data stored on the **server** for a specific user
 - The backend creates a session object and stores it
 - The server sends a unique **Session ID** to the client as a cookie
 - On each future request, the browser sends back that cookie automatically
 - The backend uses that cookie to find the session and know who the user is
- **Cookie:** A small key-value pair that the server tells the browser to store
 - Persistent Cookie: remains after browser is closed, has an expiration time
 - Session Cookie: cleared after browser is closed, doesn't have expiration time

Cookie	Session
Stored in the browser	Stored on the server
Contains a session ID	Contains user data
Sent with every request	Retrieved based on the ID in the cookie

express-session

- A **middleware** for Express.js that enables server-side session management
 - `app.use(session(...))`:
 - Each user gets a unique session ID
 - The server uses that ID to track user-specific data like login state
 - The session ID is sent to the client via cookies

Set-Cookie

- HTTP response header that tells the browser to store these cookies and send them with requests
 - HttpOnly: Prevents JS from accessing the cookie (protects against **XSS**)
 - Secure: Only sends cookie over HTTP**S** (protects against network sniffing)
 - SameSite: Controls cross-site behavior (**CSRF** protection)
 - Lax, Strict

```
app.use(session({
  secret: 'my-secret-key', // 加密签名的密钥
  cookie: {
    secure: true, // 【重点】只有在 HTTPS 下才发送 Cookie
    httpOnly: true, // 【重点】禁止 JS 读取 (防 XSS)
    sameSite: 'lax' // 【重点】限制跨站请求携带 (防 CSRF)
  }
}));
```

Attacks

- **Cross-site Request Forgery (CSRF):** Attacker tricks authenticated users to send a request without them knowing or doing anything
 - Solution: avoid cookies, SameSite cookies, **CSRF token**, double-submit cookie
- **Clickjacking:** Somebody tricks the user into clicking something that sends the request
 - Embed official website in <i-frame>, hide the buttons with your own
 - Solution: X-Frame-Options, Content Security Policy headers, User confirmation
- **Injection:** An attacker sends malicious input that's interpreted as code by the server (database, shell, etc.), allowing them to run unauthorized commands
 - **Cross-Site Scripting (XSS):** somebody injected scripts that send the request
 - Escape/encode: <script> -> <script>
 - Solutions: input validation, Use query parameterization

Attacks

Attack	Goal	Example	Defense
CSRF	Forge an unwanted user request	Fake form submits money transfer	SameSite cookies, CSRF tokens, JWT
Clickjacking	Trick user into clicking something invisible	Hidden iframe tricking user clicks	X-Frame-Options , CSP frame-ancestors
XSS	Run malicious JS in browser	Inject <code><script></code> in comments	Escaping, auto-sanitizing frameworks, CSP

Securing input & Outputs

- **Input validation:** Checking whether data received from users (forms, APIs, URLs, cookies, etc.)
 - Prevents SQL/NoSQL injection, Blocks XSS, Avoids crashing your server with unexpected types or sizes
- **Output encoding/Escaping:** Transforming user input before displaying it in HTML, JavaScript, or other output formats
 - **Escaping:** Replace dangerous characters with safe equivalents that the browser won't treat as code

Character	Escaped Form
<	<
>	>
&	&
"	"
'	'
/	/

Validation

- **Regular expressions:** patterns used to validate input formats (emails, phone numbers, IDs, etc.)

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
if (!emailRegex.test(req.body.email)) {  
  return res.status(400).send('Invalid email');  
}
```

- Validation libraries
 - Joi, Yup, express-validator

Stateful vs Stateless Authentication

- **Session-based auth (stateful):** Store login info on the server,
 - Storage: Server (Memory/Redis)
 - Session ID
- **Stateless auth:** The server doesn't store any user session or login info
 - Storage: Client (Browser)
 - Client includes all necessary identity info in each request
 - **token:** A compact, encoded string that represents a user's identity and possibly permissions
 - **JWT**, SWT, SAML

JWT

- JSON Web Token: An open standard that defines a compact and self-contained way for securely transmitting data through JSON objects
 - <Header>.<Payload>.<Signature>
 - Advantage:
 - Stateless - No server session needed
 - Self-contained - Includes all user info (no DB lookup required)
 - Secure - Signed (optionally encrypted), can set expiration (exp)
- jsonwebtoken: npm library that implements JWT
 - jwt.sign(): Create a JWT
 - jwt.verify(): Verify and decode a JWT

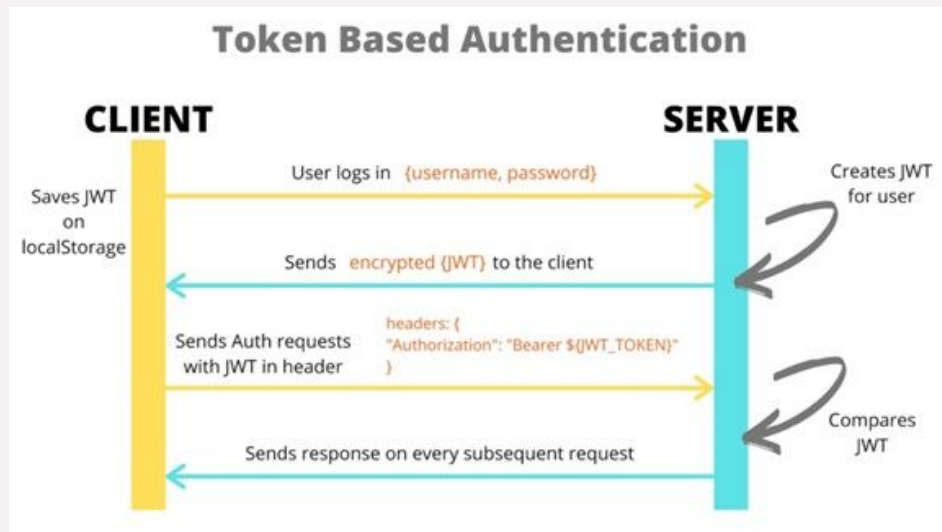
JWT Structure

- Header
 - Tells the **algorithm** and token type
- Payload
 - Contains the actual user info (claims)
 - email, role, permissions...
- Signature
 - Used to verify the token wasn't tampered with
 - Hash(Header + Payload + SecretKey)
 - Created by hashing the header + payload using a secret or private key

Encoded	Decoded
<pre>eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ij0iOnRydWV9.TjVA95OrM7E2cBab30RMhRHDcEfxjoYZgeF0Nfh7HgQ</pre>	HEADER: <pre>{ "alg": "HS256", "typ": "JWT"}</pre> PAYLOAD: <pre>{ "sub": "1234567890", "name": "John Doe", "admin": true}</pre> VERIFY SIGNATURE <pre>HMACSHA256(base64UrlEncode(header) + ".", base64UrlEncode(payload), secret)</pre>☐ secret base64 encoded

Token-Based Auth (Stateless, JWT)

- User logs in with credentials
- If valid, server generates a JWT and sends it back to the client
- Client stores JWT
 - Cookies
 - **localStorage.setItem**
 - sessionStorage
 - In-memory
- For each request, client sends the JWT in the Authorization header
 - Authorization: **Bearer** <JWT>
- Server Verifies Token



Session-based auth vs JWT-based auth

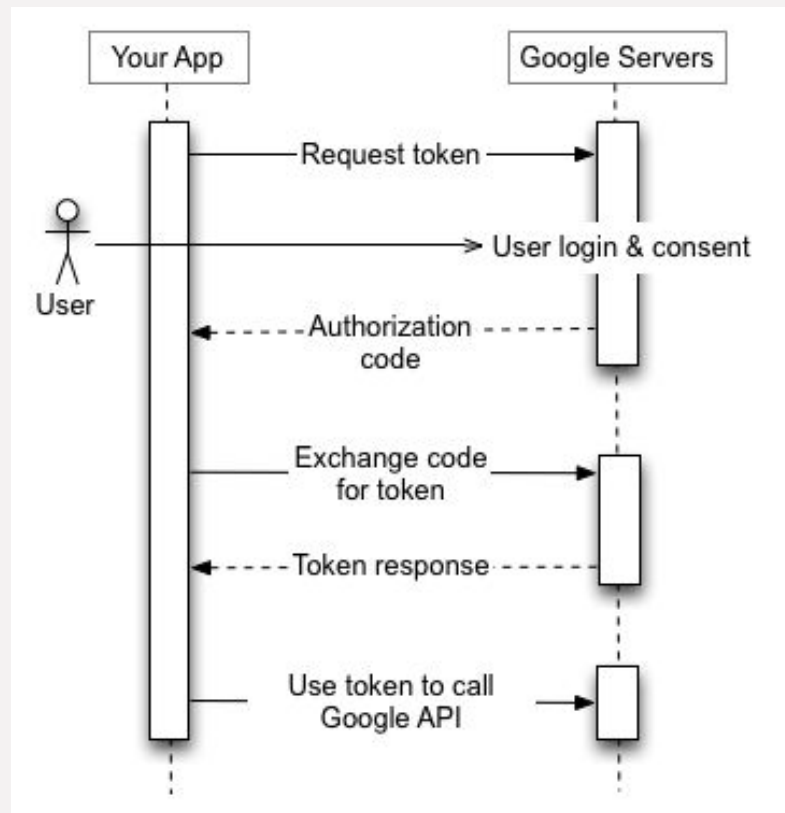
Step	Session-Based Auth 🍪	JWT-Based Auth 📄
1. Login request	User sends username/password to server	Same
2. Server verifies credentials	Yes – server checks DB	Yes – same check
3. Server creates user state	Server creates a session object (e.g., <code>{userId: 1}</code>) and stores it (in memory/Redis)	Server creates a signed JWT that includes user data (e.g., <code>{userId: 1, role: 'admin'}</code>)
4. Server sends token to client	Sends a <code>Set-Cookie</code> with the session ID	Sends the JWT (via JSON response or <code>Set-Cookie</code>)
5. Client stores token	Browser stores the session ID in a cookie	Browser stores the JWT in localStorage or cookie
6. Client sends token on next request	Browser automatically sends the session cookie	Client manually sets <code>Authorization: Bearer <token></code> header, or sends cookie if stored that way
7. Server processes request	Server reads session ID from cookie, looks it up in memory, and loads user data	Server verifies JWT signature , and extracts user data from token (no lookup needed)
8. Server authorizes user	Based on session data (e.g., <code>req.session.userId</code>)	Based on JWT payload (e.g., <code>req.user.userId</code>)
9. Logout	Server destroys session , invalidating the session ID	Client must delete the token , or server must keep a blacklist (harder)
10. Expiration / timeout	Server expires session after inactivity (e.g., 30 min)	JWT has built-in <code>exp</code> (e.g., 1h) — auto-invalidates

Session vs JWT

Feature	Session	JWT
Storage	Server-side	Client-side (token)
State	Stateful	Stateless
Scaling	Harder to scale due to server-side state management	Easier to scale (no session store needed)
Security	Safer from token theft, but vulnerable to CSRF	Vulnerable to token theft, needs secure storage and HTTPS
Token Revocation	Easy (just delete session)	Hard (needs token blacklist or short expiry)
Payload Size	Small (session ID only)	Larger (contains user data)
Use Case	Traditional web apps with server-rendered pages	Modern SPAs, mobile apps, microservices
Expiration	Controlled by server	Embedded in token, fixed expiry
Transmission	Usually via cookies	Can be sent via Authorization header or cookies

SSO & OAuth 2.0

- **Single Sign-On (SSO):** Allow users to log in once and gain access to multiple apps or systems without logging in again for each one
 - Common protocols: OAuth2, OIDC, SAML
- **Open Authorization (OAuth) 2.0:** A token-based authorization standard for allowing applications to access data from other applications
 - Requires user permission
 - Doesn't share user credentials between applications



Amazon Web Services (AWS)

- A **cloud computing** platform by Amazon that provides on-demand services like:
 - **S3** (Simple Storage Service): Stores files like images, videos, and documents
 - EC2 (Elastic Compute Cloud): Virtual servers to run your applications
 - RDS (Relational Database Service): Managed databases like MySQL, PostgreSQL
 - Lambda: Run code without managing servers ("serverless")
 - **CloudFront**: Speed up website delivery (CDN)
 - **IAM**(Identity and Access Management): Control who can access what (permissions & security)

Amazon Web Services (AWS)

服务	全称	核心功能
S3	Simple Storage Service	存文件 (图片/视频)
EC2	Elastic Compute Cloud	租电脑运行代码
RDS	Relational Database Service	存结构化数据 (SQL)
Lambda	-	运行代码函数, 按次计费
CloudFront	-	加速全球访问 (CDN)
IAM	Identity & Access Mgmt	管理用户和权限

Why AWS (Cloud)

- **Scalability:** Scale up or down quickly and easily to meet changing business needs
- **Agility:** Develop and deploy applications faster, and dynamically change your infrastructure as required
- **Reliability:** Achieve a seamless experience for your customers with the same technologies used by Amazon to run its own business
- **Security:** Protect your data more cost-effectively than on-premises with the largest set of security services and features

AWS S3

- Amazon S3 (Simple Storage Service) is Amazon Web Services' object storage system
 - S3 **bucket**: A storage container
 - Region: The geographic location of the container
 - Object: A file inside the container
 - Key: Unique identifier for an object within a bucket
 - The full "path/filename"



AWS IAM

- IAM (Identity and Access Management): The AWS service that lets you securely **control access** to AWS resources like S3, EC2, DynamoDB, etc.
 - Bucket policy: upload files to your S3 bucket
 - **AWS_ACCESS_KEY_ID**
 - unique ID that identifies IAM user/role
 - **AWS_SECRET_ACCESS_KEY**
 - A secret password to sign and authenticate AWS API requests
 - **AWS_REGION**: geographic region

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicRead",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::your-bucket-name/*"
    }
  ]
}
```

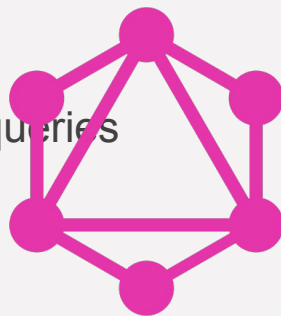
AWS CloudFront

- AWS CloudFront is a Content Delivery Network (CDN)
 - A service that delivers your website, images, videos, or API responses faster to users by caching them close to where the users are located
- Create a CloudFront distribution
- Set the origin domain name

Feature	Description
 Faster Loading	Delivers files closer to users for better speed
 HTTPS Support	Easily add SSL to your site or API
 Caching	Reduces repeated requests to your origin (e.g., S3 or EC2)
 Security	Works with AWS Shield & WAF to block DDoS or malicious requests
 Analytics	Provides metrics like traffic, cache hits, geographic origin, etc.
 Custom Domain	Use your own domain with HTTPS via AWS Certificate Manager

GraphQL

- GraphQL is a **query language** for APIs and a runtime for fulfilling those queries with your existing data
- It's an alternative to RESTful with a more flexible, client-driven approach



Benefit	Explanation
✓ Single endpoint	All requests go through <code>/graphql</code>
✓ Get exactly what you need	No underfetching or overfetching
✓ Nested queries	Request related data in one go
✓ Strong typing	Schema defines all types clearly
✓ Great for front-end devs	Frontend controls the shape of the data

Query

- A query is used to fetch (read) data
 - like GET in REST
- Returns exactly what you ask for, no more, no less
- You can request nested, related data all in one request

```
query {  
  posts {  
    id  
    title  
    body  
  }  
}
```

```
query {  
  user(id: "1") {  
    name  
    posts {  
      title  
      comments {  
        text  
        author {  
          username  
        }  
      }  
    }  
  }  
}
```

Mutation

- A mutation is used to change (write) data
 - like POST, PUT, or DELETE in REST
 - Mutate & Query

```
mutation {  
  addPost(title: "New Post", body: "Hello!", userId: 1) {  
    id  
    title  
    body  
  }  
}
```

```
mutation {  
  deletePost(id: 1) {  
    success  
    message  
  }  
}
```

```
mutation {  
  updatePost(id: 1, title: "Updated Title") {  
    id  
    title  
    body  
  }  
}
```

Subscriptions (Real-time)

- Used for real-time updates over **WebSocket**

```
subscription {  
  newPost {  
    title  
    author {  
      username  
    }  
  }  
}
```


GraphQL vs REST

Feature	GraphQL	REST
Request Shape	Client-defined	Server-defined (fixed endpoints)
Overfetching	✗ Avoided	✓ Common
Underfetching	✗ Avoided	✓ Common
Versioning	No need	Uses <code>/v1</code> , <code>/v2</code> , etc.
Data Fetching	Single request for nested resources	Multiple requests

Type System

- The Type System defines what data your API can provide
- **Scalar Types**: Built-in smallest, indivisible units of data
 - Int, Float, String, Boolean, **ID** (Unique identifier, usually a string)
- **Object Types**: custom types that represent your business logic
 - Represent a kind of object you can fetch from your service, and what fields it has
- **Modifiers**: The Rules
 - Non-Nullable: !
 - List: []

```
type User {  
  id: ID!  
  name: String!  
  email: String!  
  posts: [Post!]!  
}
```

```
type Post {  
  id: ID!  
  title: String!  
  body: String!  
  author: User!  
}
```

Schema

- Schema is the blueprint of the API, it acts as a contract between the client and server
 - What types of data your API can return or accept
 - What operations (queries, mutations, subscriptions) clients can perform
 - How those operations are structured

```
# 这就是 Schema
type User {
  id: ID!
  name: String!
  age: Int
  # 关联关系: 一个人可以有多个帖子
  posts: [Post!]!
}

type Post {
  id: ID!
  title: String!
}

# 根节点: 查询入口
type Query {
  # 承诺: 你可以查 user, 给我个 id, 我还你个 User 对象
  user(id: ID!): User
}
```

Resolver

- Functions that fetch the data for each field
 - For every field defined in the Schema, there must be a corresponding Resolver function in the backend to fetch the data

```
const resolvers = {  
  Query: {  
    // Matches schema: user(id: ID!)  
    user: (parent, args, context) => {  
      // 1. Get the ID from the frontend arguments  
      const userId = args.id;  
      // 2. Actually go to the database  
      return db.query(`SELECT * FROM users WHERE id = ${userId}`);  
    }  
  },  
  
  User: {  
    // Matches the 'posts' field inside the User type (Nested Query)  
    posts: (parent, args, context) => {  
      // 'parent' is the User object found in the step above  
      const userId = parent.id;  
      // Find posts belonging to this specific user  
      return db.query(`SELECT * FROM posts WHERE user_id = ${userId}`);  
    }  
  }  
};
```

Tech Stack

- Node.js: express-graphql, [apollo-server](#)
- Frontend: React + [Apollo Client](#)
- Database: MongoDB

Next.js



- Next.js is a React **framework**, it's built on top of React but adds powerful features, can build modern web applications faster and more efficiently
 - Full Stack Application
 - Build backend endpoints inside the same app
 - File-based **Routing**
 - Create pages by adding files in the pages/ or app/ director
 - Server-side Rendering
 - Pages can fetch data and render on the server
 - Static Rendering
 - Pages can be pre-built for speed and SEO
 - SSR + Static

Page Router vs App Router

- The Pages Router is the original routing system in Next.js using `pages/`.
- The App Router is the newer, more powerful system using **`app/`**, with layouts, server components, and modern React features
 - Nested Layouts
 - Server Components

`pages/`

```
|— index.js      → route: "/"
|— about.js     → route: "/about"
|— blog/
|   └─ [slug].js → route: "/blog/:slug"
```

`app/`

```
|— layout.tsx    → shared layout
|— page.tsx      → route: "/"
|— about/
|   └─ page.tsx  → route: "/about"
|— blog/
|   └─ [slug]/
|       └─ page.tsx → route: "/blog/:slug"
```

Page Router vs App Router

Feature	Pages Router (Old)	App Router (New)
Directory	<code>pages/</code>	<code>app/</code>
How to define a Route	<code>pages/about.js</code>	<code>app/about/page.js</code>
Default Component Type	Client Component (Hydrated)	Server Component (Zero JS sent to client)
Data Fetching	<code>getStaticProps</code> , <code>getServerSideProps</code>	Standard <code>async/await fetch()</code> inside the component
Layouts	Difficult (via <code>_app.js</code>)	Easy (via <code>layout.js</code> , supports nesting)
Loading State	Manual conditional logic	Automatic via <code>loading.js</code> file

Static Rendering (SSG)

- The HTML is generated at Build Time (when you deploy the code)
 - Usage: Blogs, Documentation, "About Us" pages...
- App Router: In the App Router, this is the default behavior

```
// app/blog/page.js
async function getPosts() {
  // Next.js runs this ONCE at build time and caches the result.
  const res = await fetch('https://api.example.com/posts');
  return res.json();
}
```

Server-Side Rendering (SSR)

- The HTML is generated at Request Time (every time a user visits)
 - Usage: User Dashboards (Private data), Social Media Feeds, Live Stock Prices...
- App Router: You must explicitly tell Next.js not to cache.

```
// app/stock/page.js
async function getStock() {
  // 'no-store' means: Don't use the cache. Fetch fresh data every time.
  const res = await fetch('https://api.example.com/stock', { cache: 'no-store' })
  return res.json();
}
```

SSG vs SSR

Feature	Static (SSG)	SSR
When is HTML made?	Build Time (Before deployment)	Request Time (When user visits)
Speed	 Fastest (Instant file delivery)	 Slower (Server must calculate)
Server Load	Low	High
Data Freshness	Old (Until next build)	Fresh (Real-time)
SEO	Excellent	Excellent
App Router Syntax	<code>fetch(...)</code> (Default)	<code>fetch(..., { cache: 'no-store' })</code>

React Server Components (RSC)

- React Server Components are React components that **run only on the server**
 - The server executes the component logic
 - The server sends only the result (the UI) to the browser
 - The component's JavaScript code stays on the server. It is never sent to the client
 - Direct Database Access
 - Stop writing API endpoints for simple data fetching

```
// app/page.tsx
// This is a Server Component by default
import db from './lib/db';

export default async function Page() {
  // Direct SQL query inside the component!
  // This code is secure; the browser never sees it.
  const users = await db.query('SELECT * FROM users');

  return (
    <div>
      {users.map(user => (
        <div key={user.id}>{user.name}</div>
      ))}
    </div>
  );
}
```

Server vs. Client

- Server Component **CANNOT**:
 - Listen to browser events
 - e.g: Click button
 - Access browser APIs
 - Fetching on the Client
 - `useState()` + `useEffect()`
 - Fetching on the Server
 - Faster Initial Page Load, caching...
 - Maintain state
 - Use effects

Server vs. Client

- If you need interactivity, you must opt-out and use a Client Component

Feature	Server Component (Default)	Client Component (<code>'use client'</code>)
Fetching Data	✓ Direct DB / File System access	✗ Must use HTTP <code>fetch</code> / API
Secrets	✓ Can use API Keys / Env Vars	✗ Never (Visible to user)
Interactivity	✗ No <code>onClick</code> , <code>onChange</code>	✓ Yes (Buttons, Forms)
State	✗ No <code>useState</code> , <code>useEffect</code>	✓ Yes
Browser APIs	✗ No <code>window</code> , <code>localStorage</code>	✓ Yes

Additional materials

- [How to deploy React App on AWS S3](#)
- [Next.js 16 Full Course](#)
- [FullStack GraphQL React Tutorial - Learn Apollo Client / Apollo Server For Beginners](#)

Homework - Short Answer Questions

1. How does HTTPS differ from HTTP?
2. Explain hashing. Why do we hash passwords and other sensitive information?
3. What's the difference between authentication and authorization?
4. Is HTTP stateful or stateless? What does it mean for it to be stateful or stateless?
5. What are sessions and cookies? Why do we use them?
6. Explain CSRF. How do we prevent it?
7. Explain clickjacking. How do we prevent it?
8. Explain XSS. How do we prevent it?
9. Explain token-based auth. Is it stateful or stateless and why?
10. What is OAuth 2.0? Describe the OAuth workflow in detail.

Homework - Short Answer Questions

11. What is JWT?
12. Where can we store JWT?
13. What are the differences between JWT and sessions & cookies for user auth?
14. What is SOP? How do we fix it?
15. What is CORS?
16. What is SSO? What are some examples?
17. I want to store data in the browser. How can I do this with React?
18. What are regular expressions?
19. What AWS services can be used to host a React frontend?
20. What are the advantages of using S3 for hosting a React app?

Homework - Short Answer Questions

21. How does CloudFront improve website performance?
22. How does AWS Amplify compare to traditional S3 + CloudFront hosting?
23. What is React Proxy?
24. What is CI/CD?
25. What is GraphQL and how does it differ from REST?
26. How do you define a schema in GraphQL?
27. What are queries and mutations in GraphQL?
28. What is a resolver in GraphQL and how does it work?
29. What are some common use cases for GraphQL subscriptions?
30. What are some best practices for designing a GraphQL schema?

Homework - Coding Questions

1. Find the second highest salary from the employees table. (employees, salary)
2. Retrieve employees whose names start with 'A'. (employees, first_name)
3. Find all employees who have the same salary as their manager. (first_name, salary, employees, manager_id, employee_id)
4. Write a JSON object representing a user with name, age, and email